

测试框架说明

1. 关于实验环境

Trace测试使用Linux系统部署测试环境，我们提供了三种实验环境供同学们选择：

(1) 实验室虚拟机

实验室的台式机均已提前安装好基于WSL2虚拟机的实验环境，推荐同学们直接使用。

WSL2虚拟机最方便的地方之一就是可以快速实现系统之间的文件拷贝。

(2) 远程实验平台

远程实验平台的资源和性能有限，不要在上课期间使用，否则服务器负载能力有限，会导致使用体验很差。

远程实验平台已经将Trace测试的运行环境部署在实验中心的服务器上，我们把所有依赖的配置都已经事先搭建完毕。无论你的电脑性能如何，无论你是在宿舍、实验室还是自习室，只要你还能连上校园网，你就能完成你的实验。具体使用方式详见[附录A：远程实验环境使用指南](#)。

温馨提示

虽然我们已经做了一些方案保证远程环境的可靠性，但在某些特殊情况下，也不能确保不出故障，为安全起见，建议同学们将代码及时上传到git仓库或者下载到本地保存。

(3) 自行部署实验环境

如果要自行安装环境，推荐安装WSL2虚拟机。安装说明见[附录B：虚拟机使用指南](#)。

感兴趣的同学也可以尝试在自己的电脑上从零开始安装实验环境，体验一下自己动手的乐趣：）具体搭建方法详见[附录C：实验环境部署指南](#)。

2. 了解测试框架

下面以WSL2虚拟机为例，介绍Trace测试框架使用方法。

首先在桌面上打开wsl2distromanager，并启动comp2008虚拟机。

在虚拟机终端输入并执行下列命令，以拉取测试框架代码：

```
1 | cd ~ && git clone https://gitee.com/hitsz-cslab/cdp-tests.git
```

关于miniLA

实现miniLA指令集的同学，拉取测试框架时，请执行以下命令：

```
1 | cd ~ && git clone -b miniLA https://gitee.com/hitsz-cslab/cdp-tests.git
```

cdp-tests目录的文件结构如下图所示。

```
.
├── bin                # 指令测试用例，用于初始化内存
│   ├── add.bin
│   ├── .....
│   ├── xor.bin
│   └── xori.bin
├── asm               # 指令测试用例的反汇编文件，用于阅读汇编代码来辅助调试
│   ├── add.dump
│   ├── .....
│   ├── xor.dump
│   └── xori.dump
├── csrc              # 测试驱动框架，包括比对逻辑
│   ├── dut.h
│   └── test.cpp
├── golden_model      # C语言编写的CPU行为模型
│   ├── emu.c
│   ├── include/...
│   └── stage/...
├── waveform          # 【运行测试后生成】的波形文件，用于查看波形来辅助调试
│   ├── add.vcd
│   ├── .....
│   └── xori.vcd
├── Makefile
├── mySoC              # 你实现的SoC的Verilog代码，放在此目录下，仅拷贝HDL代码，不拷贝IP核
│   ├── miniRV(或LA)_SoC.v
│   ├── defines.vh
│   └── .....
├── vsrc              # 仿真需要用到的其他文件
│   ├── ram.v         # Basic测试的主存模块，不含总线
│   └── bram_axi.v    # AXI测试的主存模块
├── run_all_tests.py  # 自动化测试脚本
├── Makefile
├── sim_simple.surf.ron # 单周期CPU的Surfer波形配置文件
└── sim_pipeline.surf.ron # 流水线CPU的Surfer波形配置文件
```

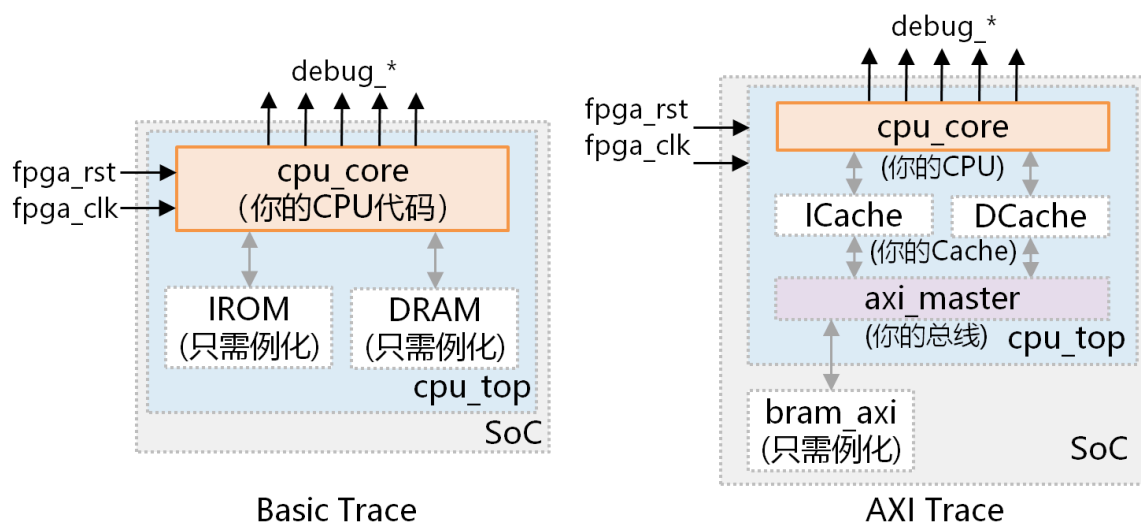
测试的原理是差分测试，即比对标准模型和待测模型之间的区别。在实验中，标准模型就是 `golden_model` 下使用C语言实现的CPU模型，而待测模块就是你所实现的CPU。驱动测试的代码逻辑位于 `csrc` 文件夹中，分别让标准模型和待测模型执行同一条指令，比对他们执行的结果，来确定你的CPU是否实现正确。你只需要关注 `mySoC` 目录，暂时不需要关注其他目录，你需要将自己实现的CPU的Verilog代码粘贴到这个目录下。

3. 添加待测试代码

`mySoC` 目录存放待测试CPU、总线控制器以及SoC顶层模块及其对外的连线等代码文件。进行测试时，需要将整个SoC工程的代码复制到该目录下。拷贝代码时，需要注意：

- 把Vivado工程目录的 `src / rtl` 下的所有源文件拷贝到 `cdp-tests / mySoC` 目录。**不要拷贝 `ip` 文件夹，也不要拷贝IP核相关文件。**
- 如果使用WSL2虚拟机，在Windows下把源文件拷贝到虚拟机之后，有时会出现许多带有 `Zone.Identifier` 后缀的文件。这些文件需要全部删除，否则测试无法运行。删除方法是在文件资源管理器的搜索框搜索 `Zone.Identifier`，然后快捷键 `^ Ctrl + A` 全选后删除。
- 保证 `mySoC` 目录下的架构是 `miniRV_SoC`（或 `miniLA_SoC`）包含 `cpu_top`，然后 `cpu_top` 包含 `cpu_core`，且这些模块的模块名、实例名，以及 `cpu_core` 的接口信号均不要改动。课程提供的模板工程默认满足要求，不要改动。
- 不要修改所有源文件中任何出现 `RUN_TRACE` 宏定义的代码。
- 不要修改所有源文件中任何在行内出现 `/* verilator public */` 的代码，并且不要删除或修改此注释。
- 在Trace测试框架中，SoC顶层模块的复位信号 `fpga_rst` 是高电平复位。
- 系统复位后首条指令的地址是 `0x00000000`。

Trace测试框架会根据SoC是否支持AXI总线，自动进行相应架构的测试，如下图所示：



Basic Trace只会测试CPU内核，而AXI Trace会将CPU连同ICache、DCache和总线一起测试。



Trace测试框架只会测试主存访问，不会测试外设访问

如果SoC通过了AXI Trace测试，但下板测试失败，则首先应排查Uncached访问和外设访问的实现是否存在问题。

Trace测试框架使用 `cpu_core.v` 末尾的两组Trace信号来获取待测试CPU的部分结构状态。其中，第一组带有 `debug_wb_` 前缀的信号用于获取写回寄存器的信息，第二组带有 `debug_mem_` 前缀的则用于获取写访存的信息。单周期CPU的demo工程已提前连接好两组Trace信号：

`cpu_core.v`

```
252 wire [31:0] debug_wb_pc /* verilator public */ ; // WB阶段的PC
253 wire debug_wb_rf_we /* verilator public */ ; // WB阶段的寄存器写使能
254 wire [ 4:0] debug_wb_rf_wR /* verilator public */ ; // WB阶段的目标寄存器
255 (若wb_rf_we为0, 此项可为任意值)
256 wire [31:0] debug_wb_rf_wD /* verilator public */ ; // WB阶段写入寄存器的值
257 (若wb_rf_we为0, 此项可为任意值)
258
259 wire [31:0] debug_mem_pc /* verilator public */ ; // MEM阶段的PC
260 wire [ 3:0] debug_mem_we /* verilator public */ ; // MEM阶段写访存时的写使
261 能
262 wire [31:0] debug_mem_waddr /* verilator public */ ; // MEM阶段写访存时的写地
263 址 (若mem_we为0, 此项可为任意值)
264 wire [31:0] debug_mem_wdata /* verilator public */ ; // MEM阶段写访存时的写数
265 据 (若mem_we为0, 此项可为任意值)
266
267 assign debug_wb_pc = pc;
268 assign debug_wb_rf_we = rf_we1;
269 assign debug_wb_rf_wR = rf_wR;
270 assign debug_wb_rf_wD = rf_wD;

assign debug_mem_pc = pc;
assign debug_mem_we = daccess_wen;
assign debug_mem_waddr = daccess_addr;
assign debug_mem_wdata = daccess_wdata;
```

在开发流水线CPU及SoC时，请参照上述代码中的注释，自行连接两组Trace信号。注意，对于同一条指令，每一组信号只能有效一个时钟周期，特别是 `debug_wb_rf_we` 和 `debug_mem_we`。

❓ 对于流水线，一定是连接WB阶段、MEM阶段的信号吗？

不一定。第一组Trace信号的作用是获取写回寄存器的信息，而第二组则是为了获取写访存的信息。因此，你的流水线CPU在哪个阶段写回寄存器，就把哪个阶段的信号连接到第一组Trace信号。同理，流水线CPU在哪个阶段发出写访存请求，就把哪个阶段的信号连接到第二组Trace信号。

❓ 运行AXI Trace测试时，第二组Trace信号没报错，代表Store指令的写访存操作一定没问题吗？

不一定。第二组Trace信号只能帮助Trace测试框架判断CPU发出的写请求是否正确，但写请求后续传输到DCache、总线、存储器的路径上都有可能出错，导致写访存失败，从而数据没有被正确写入主存。出现这种情况时，Trace测试会在后续相同访存地址的Load指令处报错。

若运行Basic Trace测试，需要保证代码中实例化了 IROM、DRAM（模板工程默认已实例化）：

Inst_ROM.v, Data_RAM.v

```
IROM U_irom (          // 模块名必须是IROM
    .clka    (...),
    .addra   (...),
    .douta   (...),
);

DRAM U_dram (          // 模块名必须是DRAM
    .clka    (...),
    .addra   (...),
    .douta   (...),
    .wea     (...),
    .dina    (...),
);
```

若运行AXI Trace测试，需要保证代码中实例化了 bram_axi：

miniRV_SoC.v, miniLA_SoC

```
bram_axi U_bram (          // 模块名必须是bram_axi
    .s_aclk      (...),
    .s_aresetn   (...),
    .s_axi_awid   (...),
    .s_axi_awaddr (...),
    .s_axi_awlen  (...),
    .s_axi_awsz   (...),
    .s_axi_awburst (...),
    .s_axi_awready (...),
    .s_axi_awvalid (...),
    .s_axi_wdata  (...),
    .s_axi_wstrb  (...),
    .s_axi_wvalid (...),
    .s_axi_wlast  (...),
    .s_axi_wready (...),
    .s_axi_bready (...),
    .s_axi_bresp  (...),
    .s_axi_bvalid (...),
    .s_axi_arid   (...),
    .s_axi_araddr (...),
    .s_axi_arlen  (...),
    .s_axi_arsz   (...),
    .s_axi_arburst (...),
    .s_axi_arready (...),
    .s_axi_arvalid (...),
    .s_axi_rdata  (...),
    .s_axi_rvalid (...),
    .s_axi_rlast  (...),
    .s_axi_rready (...),
    .s_axi_rresp  (...),
);
```

把待测试代码拷贝到 `cdp-tests / mySoC` 目录后，在虚拟机终端执行下列命令进入 `cdp-tests` 目录：

然后继续在终端执行 `make` 命令，从而编译代码，生成可执行的仿真模型。

4.1 运行单个测试

每个测试用例的名称都和某条指令的名称相同，但测试用例内部使用了多条不同的指令。

```
hubohan@laptop ~/cdp2021/tests $ make run TEST=sltu
make[1]: Entering directory '/home/hubohan/cdp2021/tests/obj_dir'
make[1]: Nothing to be done for 'default'.
make[1]: Leaving directory '/home/hubohan/cdp2021/tests/obj_dir'
[mycpu] Resetting ...
[INFO] Instruction ROM initialized with meminit.bin
[INFO] Data RAM initialized with meminit.bin
[mycpu] Reset done.
[difftest] Test Start!
ECALL at PC = 0x000004f0, Stop now.
Test Point Pass!
```

打印出Test Point Pass之后，就代表这条指令测试通过了。

如果发生了错误，就会打印如下所示的信息：

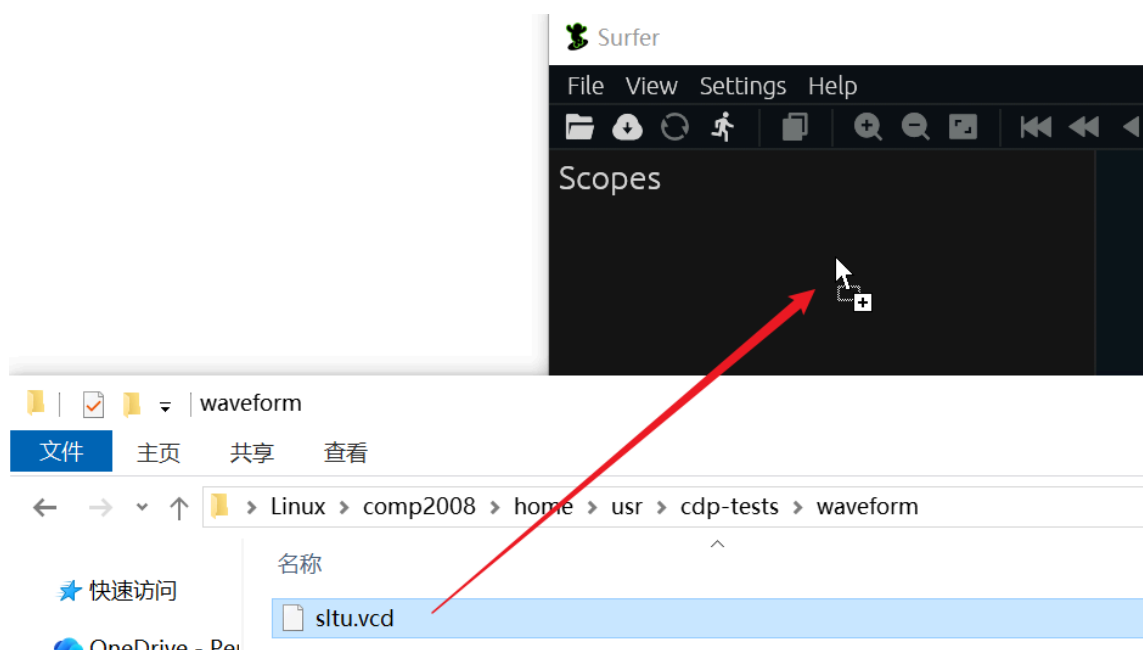
```
hubohan@laptop ~/cdp2021/tests $ make run TEST=auipc
make[1]: Entering directory '/home/hubohan/cdp2021/tests/obj_dir'
make[1]: Nothing to be done for 'default'.
make[1]: Leaving directory '/home/hubohan/cdp2021/tests/obj_dir'
[mycpu] Resetting ...
[INFO] Instruction ROM initialized with meminit.bin
[INFO] Data RAM initialized with meminit.bin
[mycpu] Reset done.
[difftest] Test Start!
[difftest] Test Failed!
===== Difference =====
SIGNAL NAME      REFERENCE      MYCPU
debug_wb_pc      0x00000008      0x00000008
debug_wb_ena            1              1
debug_wb_reg           10             10
debug_wb_value 0x00002008      0x00002000
make: *** [Makefile:12: run] Error 255
```

左栏为参照的正确实现，右栏为你实现的CPU给出的信号，通过比对这两组信号，你可以很快地确定错误发生在哪一条指令执行过程中，然后通过反汇编和波形的形式进行调试。

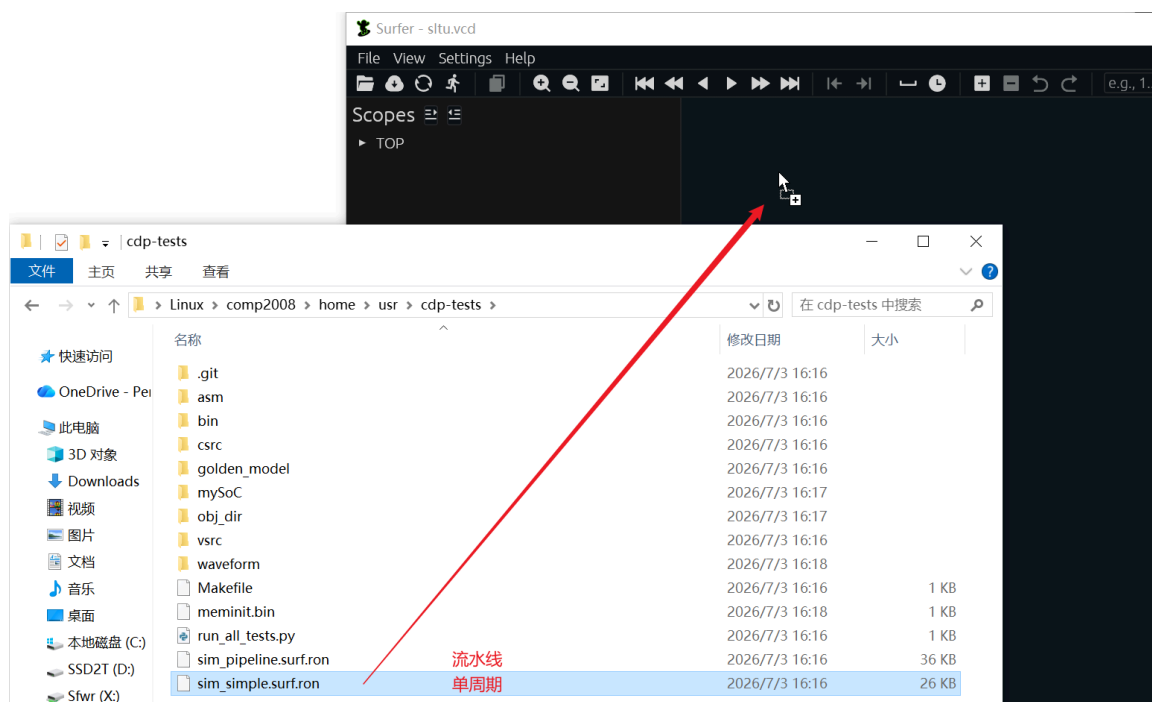
4.2 查看波形

运行某个测试之后，测试框架会在 waveform 文件夹下生成对应的.vcd波形文件。.vcd格式的波形文件可以使用GTKWave、Surfer等常用的波形查看器查看。

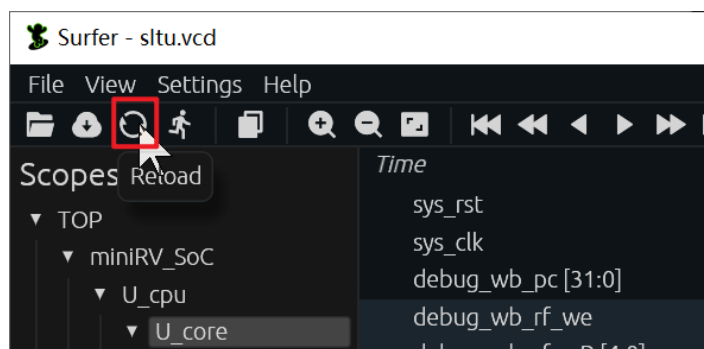
打开桌面上的Surfer工具，然后直接把 waveform 文件夹下的.vcd波形文件拖拽到Surfer，如下图所示。



然后把 `cdp-tests` 目录下的 `.surf.ron` 格式的波形配置文件拖拽到 Surfer，如下图所示。



修改HDL代码并重新编译、重新运行测试后，只需点击左上角工具栏的刷新按钮，即可查看最新的波形，如下图所示。



更多Surfer工具的使用方法，请查看[附录D. Surfer使用指南](#)。

4.3 查看反汇编

反汇编文件在 `asm` 文件夹下。在上述的 `auipc` 的测试例子中，可见在PC等于0x8处出现了错误，写回值有误。通过查看和分析 `auipc.dump` 中的反汇编指令，可以帮助我们找到出错的指令。


```

2 auipc:      file format elf32-littleriscv
3
4
5 Disassembly of section .text.init:
6
7 00000000 <_start>:
8   0:  0040006f          j    4 <reset_vector>
9
10 00000004 <reset_vector>:
11   4:  00000013          nop
12   8:  00002517      auipc   a0,0x2
13  c:  71c50513      addi    a0,a0,1820 # 2724 <_end+0x724>
14 10:  004005ef      jal    a1,14 <reset_vector+0x10>
15 14:  40b50533      sub    a0,a0,a1
16 18:  000023b7      lui    t2,0x2

```

请根据出错点，结合波形和反汇编代码，完成调试。

4.4 批量运行测试

`cdp-tests` 目录下提供 `run_all_tests.py` 的脚本来进行批量测试。

首先确保已执行过 `make` 命令，且编译没有报错，然后执行下列命令：

```
1 python3 run_all_tests.py
```

脚本运行结束后，会在最末尾输出测试总结报告。该报告会列出当前代码已通过和未通过的测试数量及测试名称。对于未通过的测试，请结合 `waveform` 目录下的波形和 `asm` 目录下的反汇编代码进行调试。

通过Basic Trace测试的示意图：

```

[INFO] Data RAM initialized with meminit.bin
[INFO] Instruction ROM initialized with meminit.bin
[mycpu] Reset done.
[difftest] Test Start!
ECALL at PC = 0x000001e8, Stop now.
Test Point Pass!
===== andi END =====

===== Testing slli =====
Peripheral name: MONITOR      base: 0x80000000      addr len: 0x00000008
Peripheral name: Digit base: 0xfffff000      addr len: 0x00000004
[mycpu] Resetting ...
[INFO] Data RAM initialized with meminit.bin
[INFO] Instruction ROM initialized with meminit.bin
[mycpu] Reset done.
[difftest] Test Start!
ECALL at PC = 0x000002ac, Stop now.
Test Point Pass!
===== slli END =====

===== SUMMARY (Basic Tests) =====
Passed Tests (45):
slt, srli, bgeu, lw, start, ori, addi, rem, slti, lhu, add, jalr, jal, sb, or, sh, sltiu, mulhu, lh, bltu, mul, beq, bne,
xori, srl, mulh, xor, sub, remu, lb, div, srai, bge, sra, divu, auipc, blt, sll, lbu, sltu, lui, and, sw, andi, slli
Failed Tests (0):

```

通过AXI Trace测试的示意图：

```
Peripheral name: Digit  base: 0xfffff000      addr len: 0x00000004
[mycpu] Resetting ...
[INFO] Instruction ROM initialized with meminit.bin
[mycpu] Reset done.
[difftest] Test Start!
ECALL at PC = 0x000001e8, Stop now.
Test Point Pass!
===== andi END =====

===== Testing slli =====
Peripheral name: MONITOR  base: 0x80000000      addr len: 0x00000008
Peripheral name: Digit  base: 0xfffff000      addr len: 0x00000004
[mycpu] Resetting ...
[INFO] Instruction ROM initialized with meminit.bin
[mycpu] Reset done.
[difftest] Test Start!
ECALL at PC = 0x000002ac, Stop now.
Test Point Pass!
===== slli END =====

===== SUMMARY =====
Passed Tests (45):
slt, srli, bgeu, lw, start, ori, addi, rem, slti, lhu, add, jalr, jal, sb, or, sh, sltiu, mulhu, lh, bltu, mul, beq, bne
, xori, srl, mulh, xor, sub, remu, lb, div, srai, bge, sra, divu, auipc, blt, sll, lbu, sltu, lui, and, sw, andi, slli
Failed Tests (0):
```

A. 远程平台使用指南

 Warning

严禁使用远程实验环境做与教学无关事宜，杜绝使用公共资源挖矿等，一经发现将严肃处理，感谢配合！

1. 速查信息

远程实验平台IP地址： **10.249.12.98** ，端口号： **6666**

 Info

用户名和初始密码都是 **你的学号** ，出于安全考虑，建议你初次登录后修改初始密码。

远程实验平台共有8个计算节点，每一个计算节点都提供相同的服务，可能你每次登录的计算节点会不一样，但无须担心，你看到的文件都是一样的。

```
===== 欢迎登录操作系统公共实验平台 =====
      节点列表      当前负荷
-----
      计算节点0      0.12%
      计算节点1      0.00%
      计算节点2      0.00%
      计算节点3      0.03%
      计算节点4      0.00%
>>> 计算节点5      0.10% <<<
      计算节点6      0.00%
      计算节点7      0.00%
=====
如果遇到问题，请寻求助教或老师的帮助。
=====
```

2. 快速入门

2.1 环境准备

需要配合远程实验平台使用的软件：

- MobaXterm

⚠ 不要用VSCode连接远程平台！

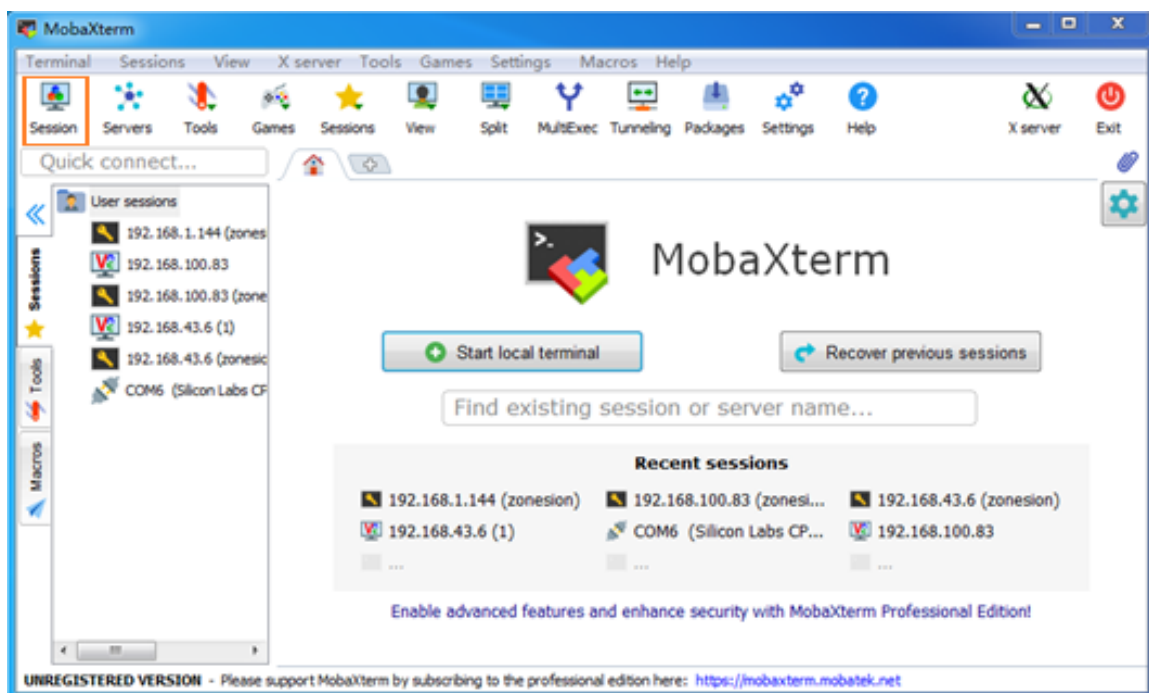
由于服务器资源有限，请尽量不使用VSCode连接远程平台（回想一下计算机系统实验的远程平台使用体验）。

2.2 使用MobaXterm

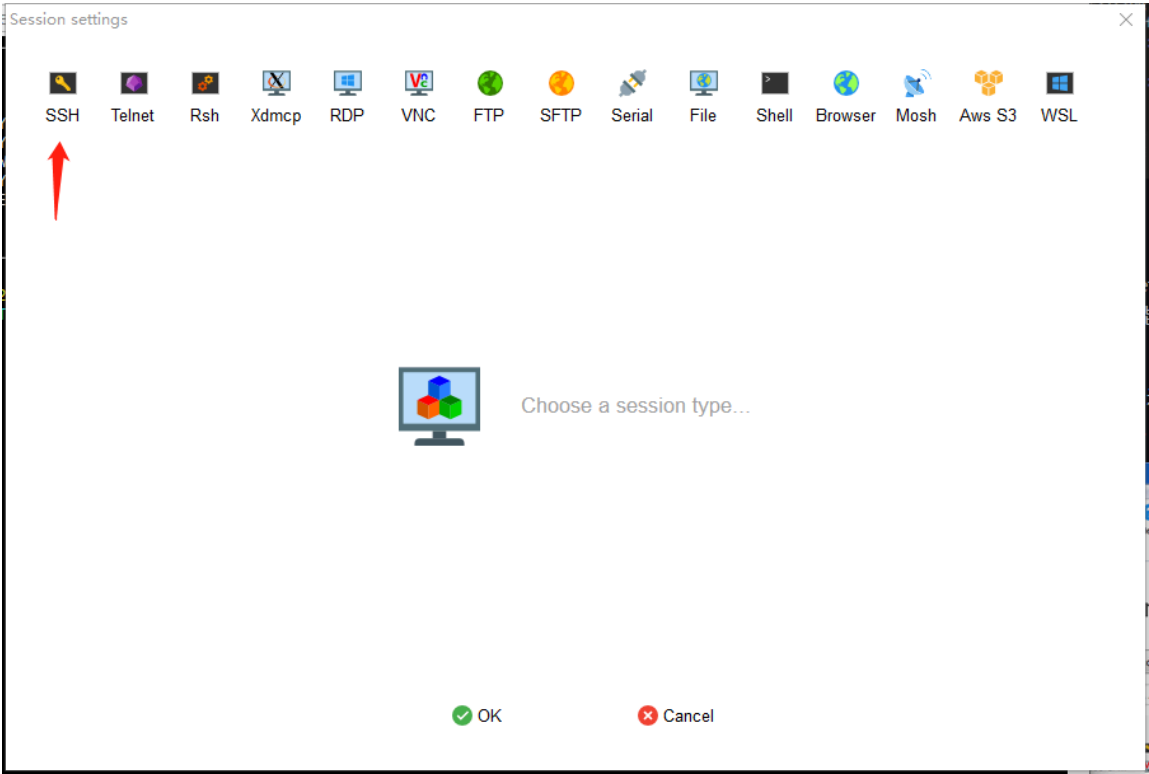
2.2.1 SSH登录

MobaXterm 是一款远程控制工具，支持创建SSH、Telnet、Rsh、Xdmc、RDP、VNC、FTP、SFTP、串口(Serial COM)、本地Shell、Mosh、Aws、WSL(微软子系统)等超多连接功能。MobaXterm 提供了人性化的操作界面，支持与主流的操作系统连接进行控制和管理操作，功能十分强大。

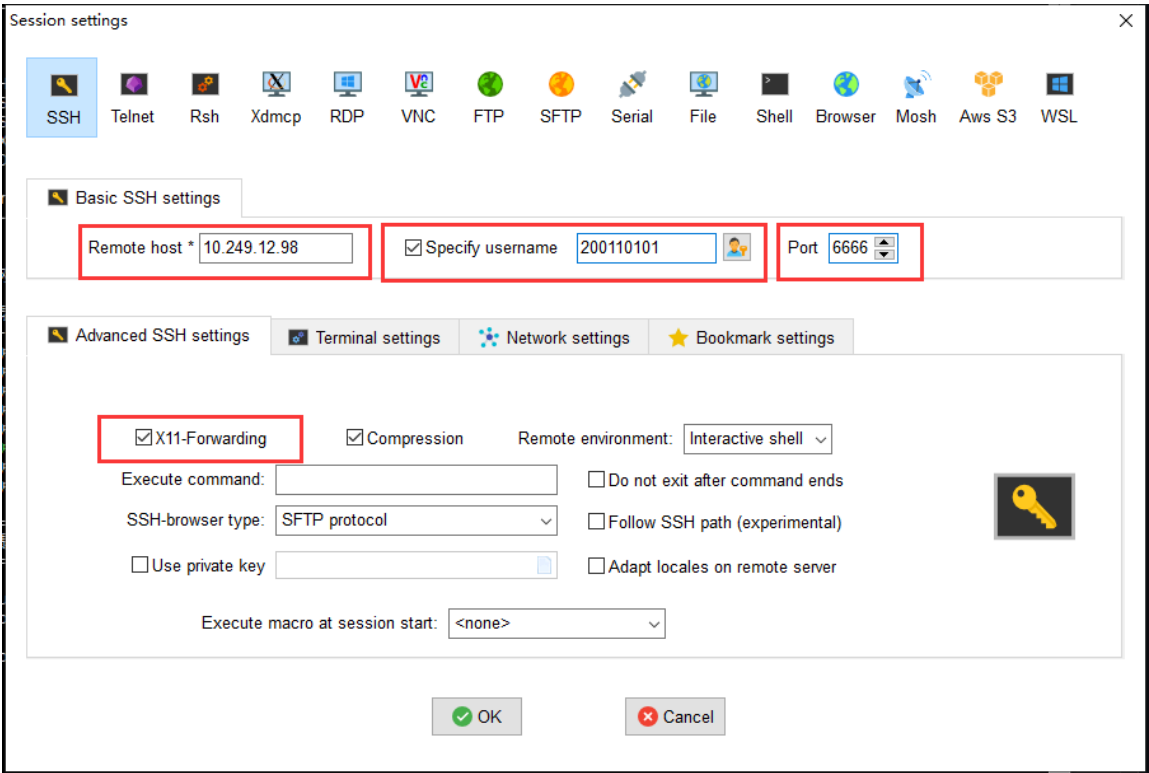
打开MobaXterm，单击打开位于左上角的工具 `Session`，选择 `New sessions` 打开新建窗口，选择 `SSH`，输入远程服务器的IP地址、登陆用户名及端口（端口号：**6666**），点击OK。



在弹出的对话框中，选择SSH。



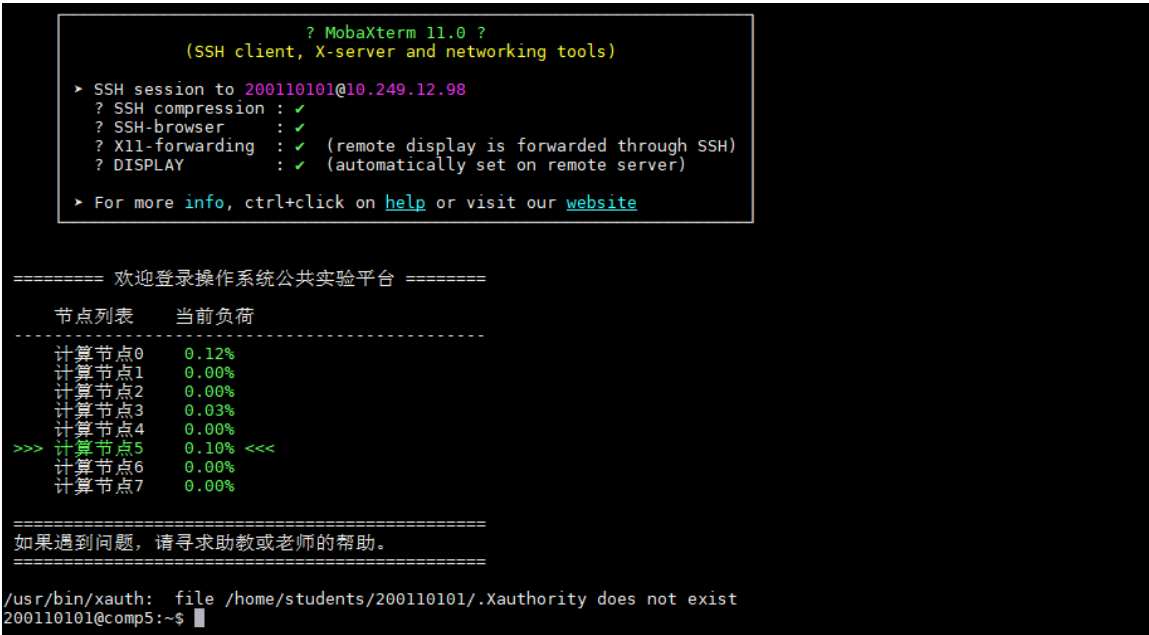
SSH配置信息如下：



输入密码：

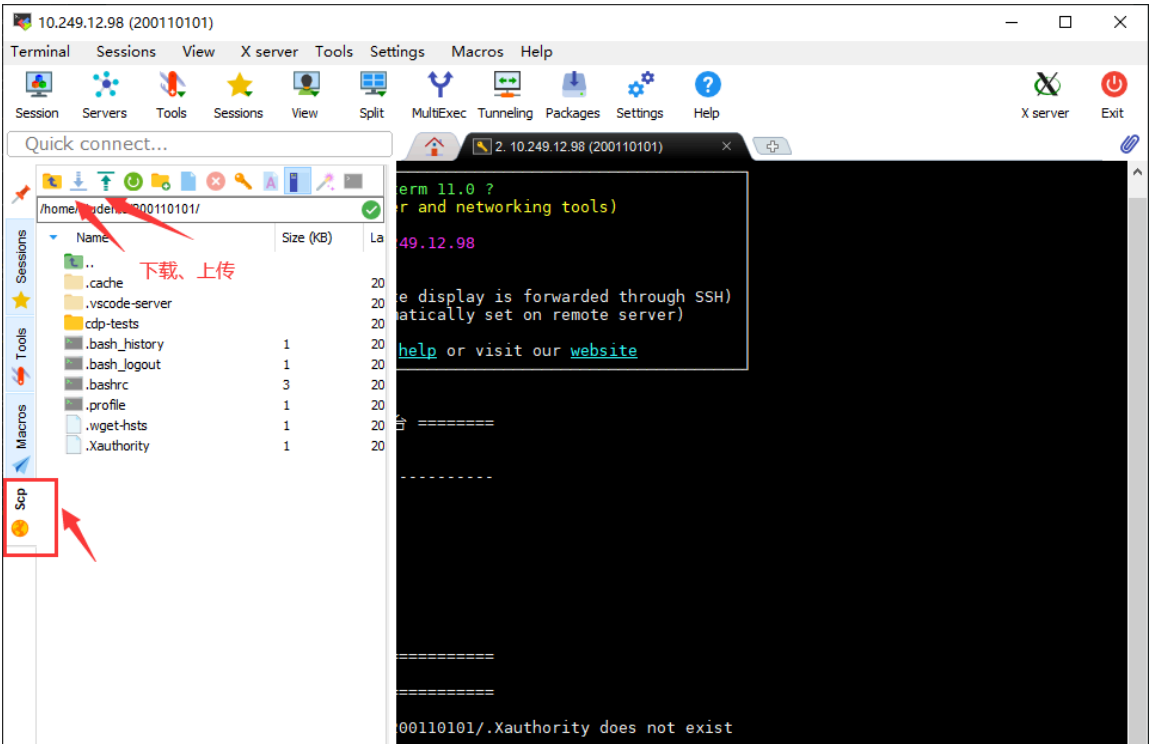


进入终端。



2.2.2 FTP文件传输

SSH远程登录完毕，双击左下角的 sftp，就出现了远程服务器的文件目录。



接下来在可以点击按钮 Upload to current folder 或者 Download selected files，在 Windows系统和远程实验平台之间上传或下载文件。

⚠️ 注意事项 🗣️

使用完毕后，需在MobaXterm中输入 `exit` 命令退出！

3. 一些进阶指引

3.1 使用ssh密钥快捷认证

有没有感觉每次都要输入密码非常麻烦？ssh密钥可以让认证流程自动化，大大解放双手。在 **本地** Windows下，以管理员身份打开CMD，输入 `ssh-keygen`。

跟随指引，一路回车，便可完成ssh密钥的生成。**请务必保存好自己的私钥，不要泄露！**

```
C:\Users\willson>ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (C:\Users\willson\.ssh\id_rsa): C:\Users\willson\.ssh\id_rsa_priv_20210909
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\willson\.ssh\id_rsa_priv_20210909.
Your public key has been saved in C:\Users\willson\.ssh\id_rsa_priv_20210909.pub.
The key fingerprint is:
SHA256:/4Zb+ZZNlbfyuiUa9dh1o17NymyJ2idpyolyFglYJ30 willson@willson-desktop
The key's randomart image is:
+----[RSA 3072]-----+
|
|   .
|  o o E
| o o .
| . . .o
|  .S. ..*
| o. .++=+
| ..o+=+0o
| . oo.*0=0o.
| +. *B+BB
+----[SHA256]-----+
```

私钥路径
公钥路径

接下来，我们将私钥加入到自己的身份认证中。在终端继续输入：

- 1|ssh-add 你的私钥的完整路径
- 2|# 例如，如上图所示，我应该键入ssh-add C:\Users\willson\.ssh\id_rsa_priv_20210909

在你的Windows上将公钥文件打开（用你喜欢的纯文本编辑器，比如记事本或者VS Code），复制其中的内容。

最后，登录你想要快捷认证的计算节点的终端，并执行下面的命令：

- 1|mkdir -p ~/.ssh
- 2|vi ~/.ssh/authorized_keys

这会打开一个文件编辑界面。按下 `i` 按键，进入编辑模式，并粘贴到上面节点终端中编辑界面的一个新行里。按下 `Esc`，按下 `:wq` 可保存并退出。如果不放心，可以 `cat ~/.ssh/authorized_keys` 并检查输出和你刚才贴进去的是不是一样的。配置完成后，下次你从Windows登陆时，应该就无需密码认证了。

4. Q&A

4.1 我不是Windows 10系统，这会带来什么问题吗？

4.1.1 我在使用Linux/macOS？

Visual Studio Code也有Linux版和macOS版，具体操作大同小异，毕竟环境主要在服务器上。

4.1.2 我正在使用旧的Windows版本/我没有更新我的Windows 10系统？

请更新你的系统。使用过时的系统有各种各样的风险，其中最大的是与时代脱轨。

4.1.3 我正在使用Windows 11？

此教程完全兼容Windows 11。

4.2 我输入的所有命令看起来都是对的，但是为什么运行失败了？

确保你输入的是正常的半角英文字符与英文标点符号，如图所示：



```
C:\Users\willson> s s h
's s h' 不是内部或外部命令，也不是可运行的程序
或批处理文件。

C:\Users\willson>ssh
usage: ssh [-46AaCfGgKkMNnqsTtVvXxYy] [-B bind_interface]
           [-b bind_address] [-c cipher_spec] [-D [bind_address:]port]
           [-E log_file] [-e escape_char] [-F configfile] [-I pkcs11]
           [-i identity_file] [-J [user@]host[:port]] [-L address]
           [-l login_name] [-m mac_spec] [-O ctl_cmd] [-o option] [-p port]
           [-Q query_option] [-R address] [-S ctl_path] [-W host:port]
           [-w local_tun[:remote_tun]] destination [command]
```

这可以通过一起按下shift+空格，或者切换到英文输入法解决。

永远不要在中文输入法下输入命令！

B. 虚拟机使用指南

1. 下载虚拟机

测试环境及相关工具下载地址：

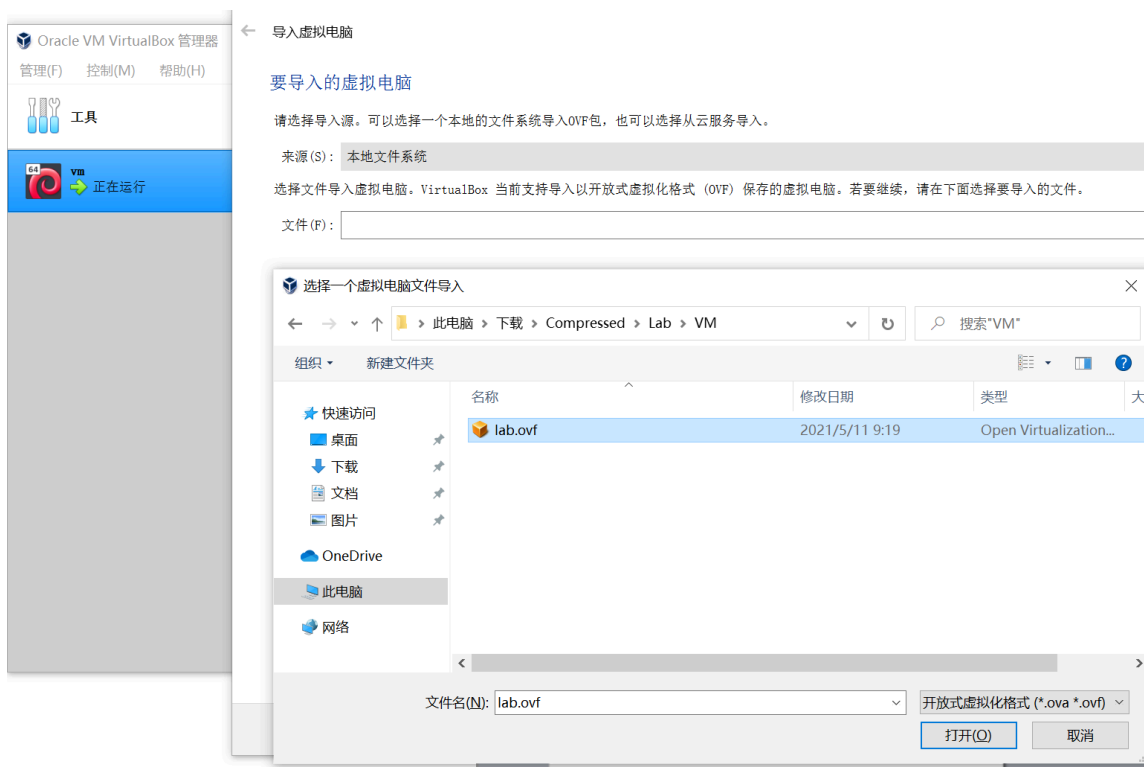
- WSL2虚拟机镜像：[comp2008_wsl2.tar](#)
- Ubuntu虚拟机镜像：[vm.ova](#)
- MobaXTerm和VirtualBox：[虚拟机工具.zip](#)

如果下载的是WSL2虚拟机，则自行搜索如何安装WSL2环境，然后导入虚拟机、启动虚拟机即可使用。

如果下载的是Ubuntu虚拟机，请参照本页后的“附录”导入虚拟机。

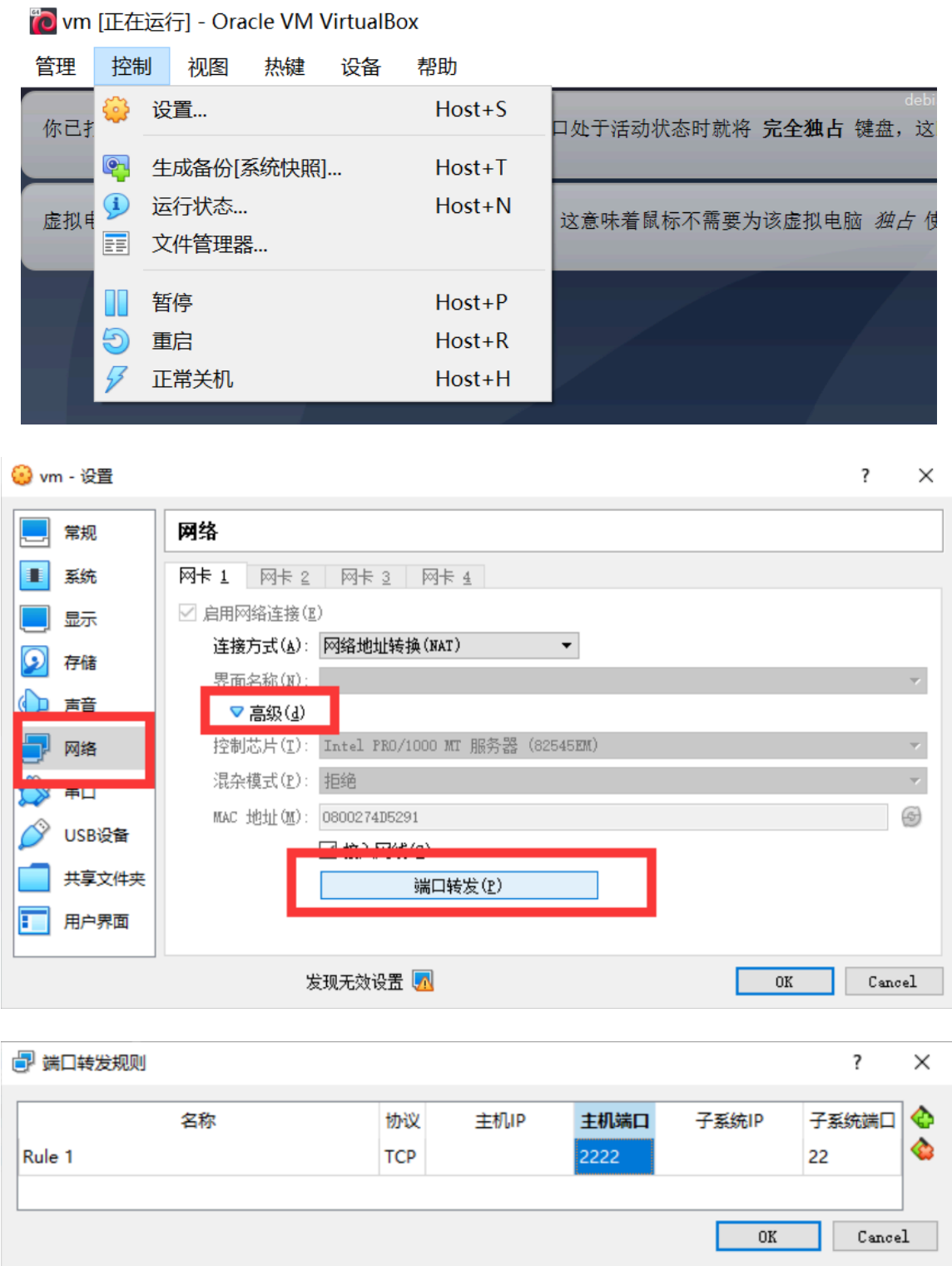
2. 导入虚拟机

推荐使用VirtualBox。下载完毕后，请在Virtualbox中导入：管理->导入虚拟电脑，选择对应OVF文件。



3. 配置虚拟网卡

VirtualBox中对于虚拟网卡，需要稍微配置一下才可以使用，具体的配置如下：

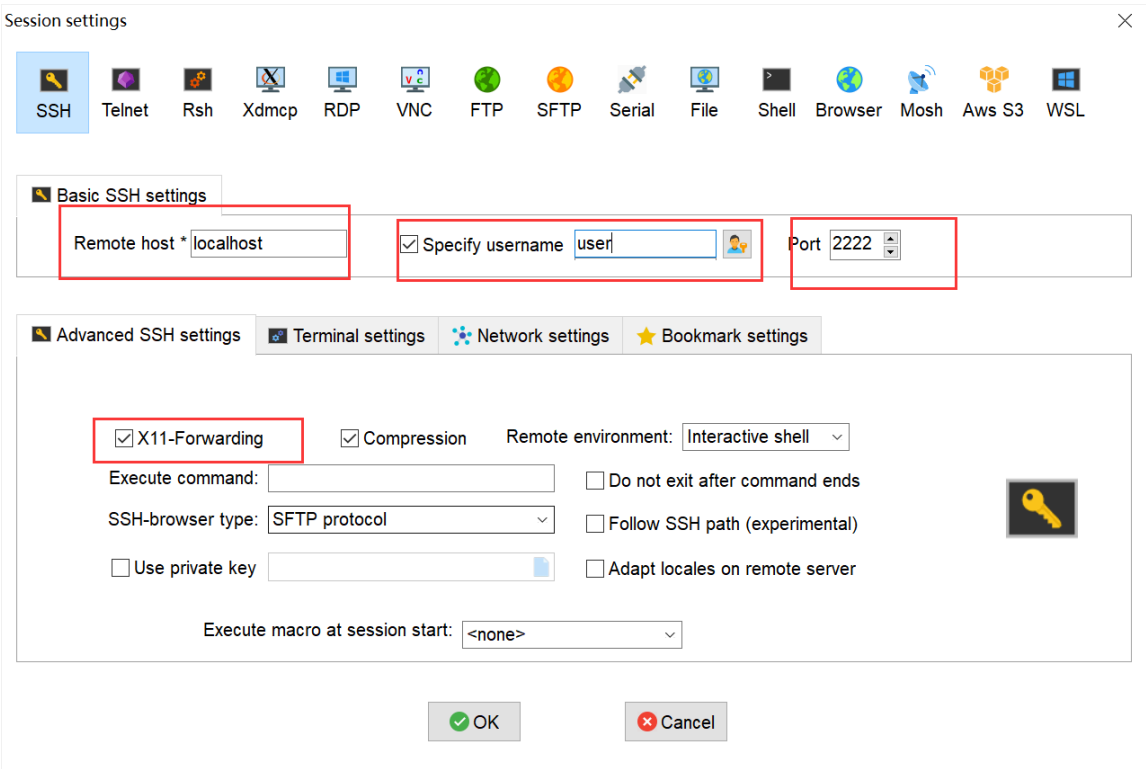


4. 使用MobaXTerm

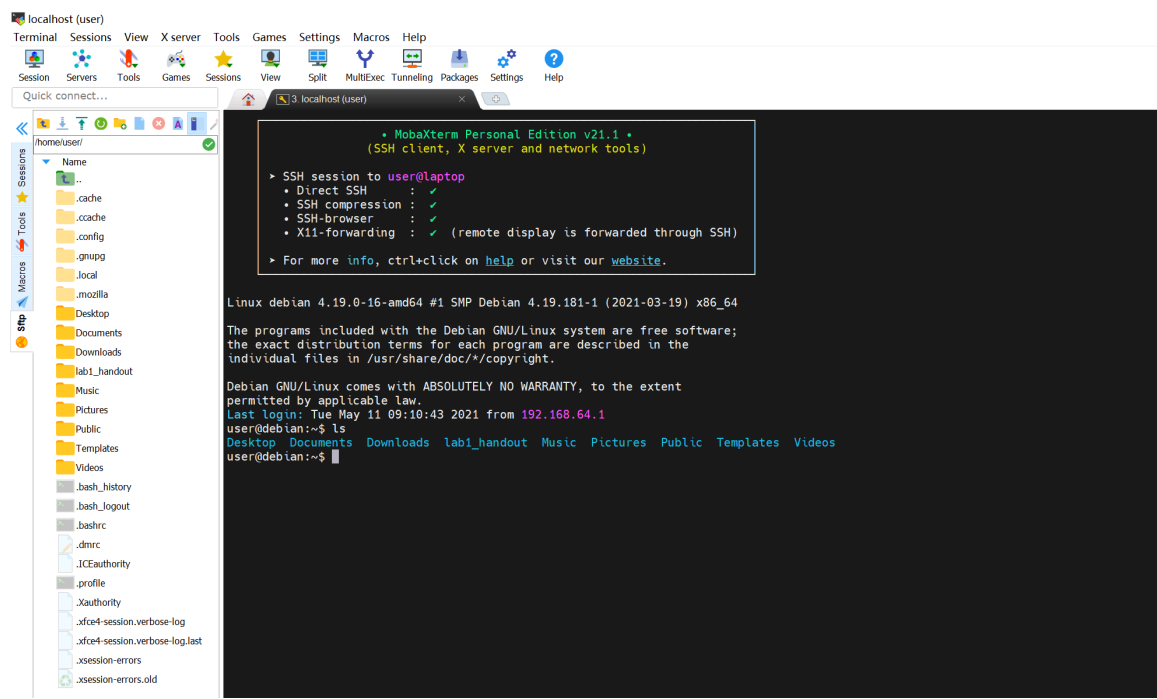
为了更好的使用体验，推荐使用MobaXTerm通过ssh连接到本地的虚拟机进行进一步的操作。压缩包里提供了MobaXTerm。

在MobaXTerm中，点击左上角“会话”。

在弹出窗口中，点击”SSH“，按如下内容填写（“远程主机”地址需要自己查询）。填写完毕后，点击“好 的”。输入虚拟机密码123456。



连接完毕之后，虚拟机可最小化到后台，在命令行中进行操作。左侧可以与虚拟机的文件交互（上传、下载），右侧可以输入命令。



C. 实验环境部署指南

Info

虽然我们提供了远程实验平台或完整的虚拟机环境，但同学们如果学有余力，也可以自己动手搭建实验环境，这样你得到的收获才是最大的：)

也许你可能会碰到非常多的问题，但不用担心，你遇到的问题，在互联网上大概率都能找到解决方案。

实验工具

1. 你需要安装一个Linux操作系统。它可以是一个能运行Linux系统的虚拟机，也可以是在你自己电脑上启用WSL（Windows Subsystem for Linux），或是整一台运行Linux的电脑。
2. 安装C++编译器和make工具。
3. 安装verilator。不建议大家使用apt-get安装，因为这样安装的版本较老。安装方法请参考[Verilator官方安装指南](#)。

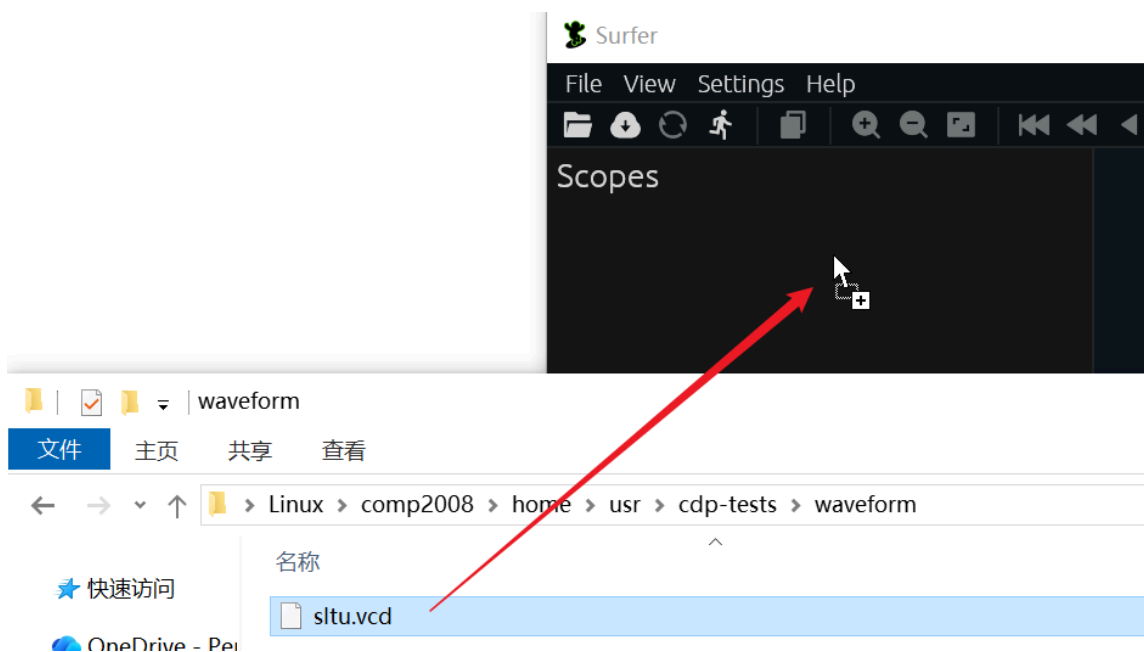
安装完成后，请参考远程实验平台或虚拟机用法，下载cdp-tests测试包，编译运行，查看测试结果。

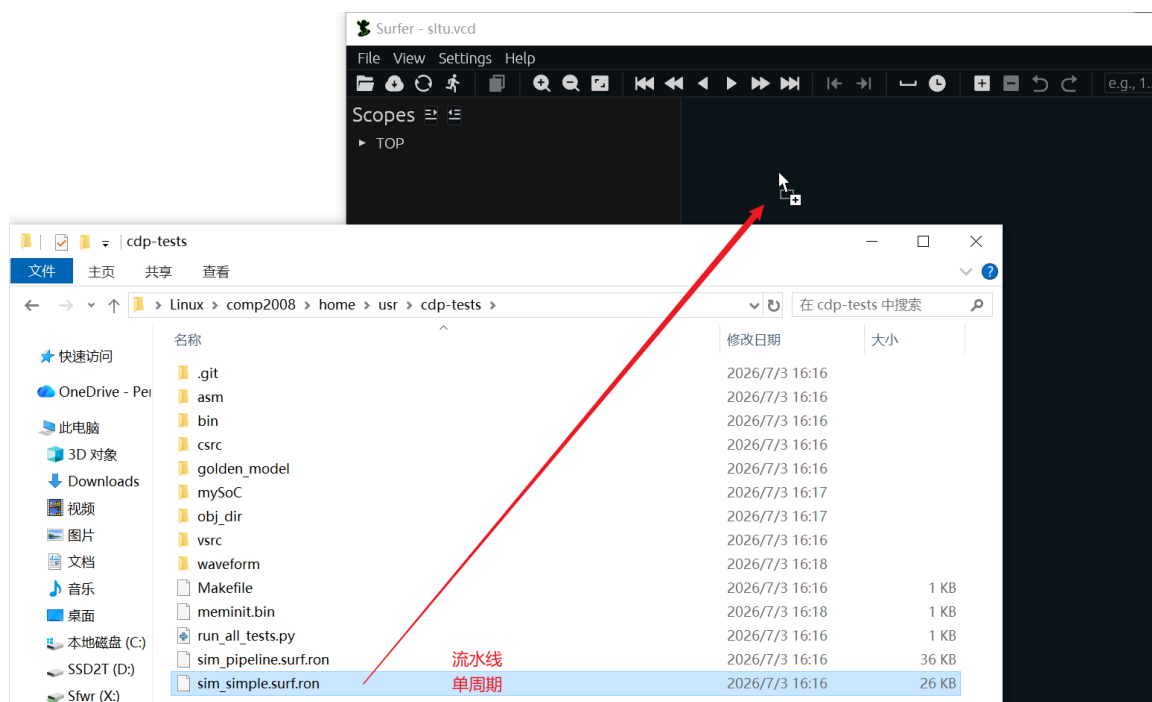
D. Surfer使用指南

Surfer是一个开源、可扩展、跨平台的数字电路波形查看工具。

1. 打开文件

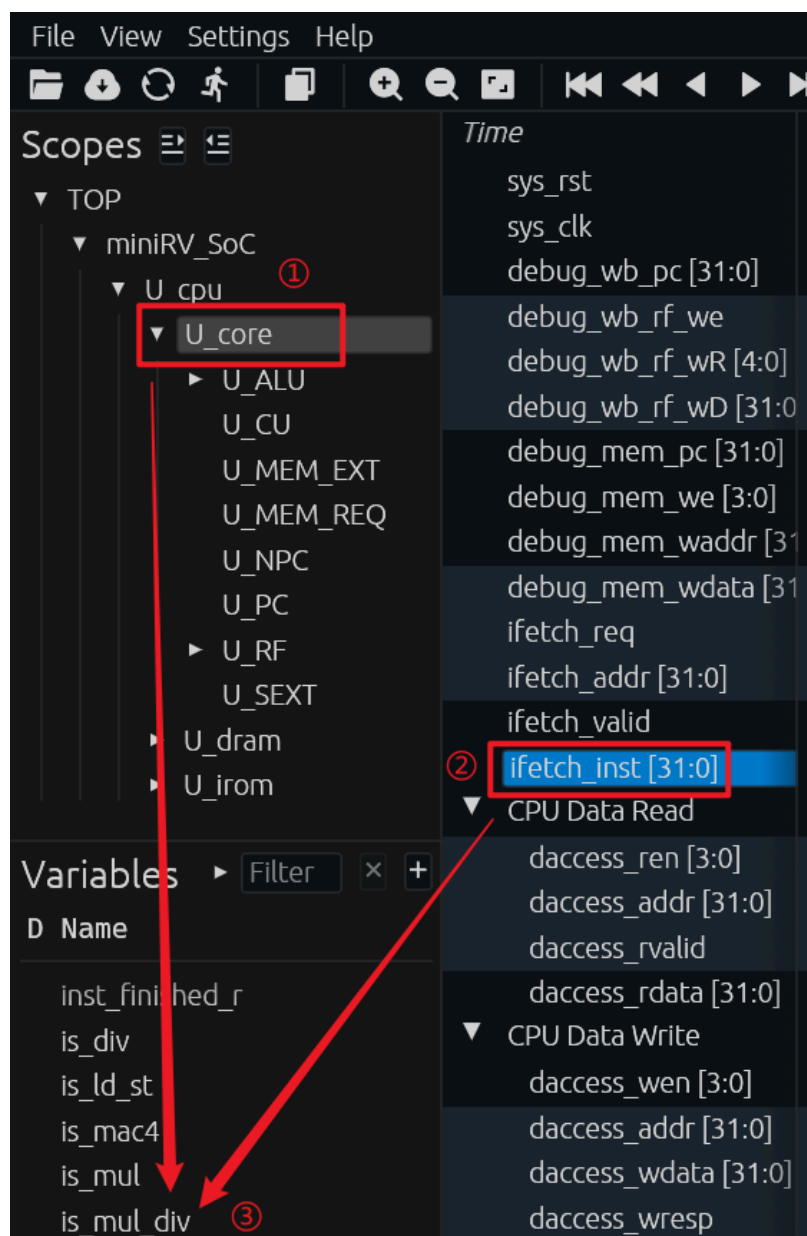
波形文件和配置文件都可以直接拖拽打开，如下图所示。





2. 添加和删除信号

首先在左上方的Scopes区域找到想要查看信号的模块，单击选中；然后右侧选择想要把信号添加到哪个信号下面；最后再点击Variables区域下的信号，如下图所示。



只要鼠标点击Variables内的信号，这些信号就会被添加到右侧被选中的信号下方。比如在上图中，如果点击Variables内的 `is_mul_div` 信号，则该信号就会被添加到右侧的 `ifetch_inst` 信号的下方。

建议先在右侧选好希望添加信号的位置，再点击Variables内的信号，否则容易使得波形中的信号杂乱无章，不利于分析和调试。

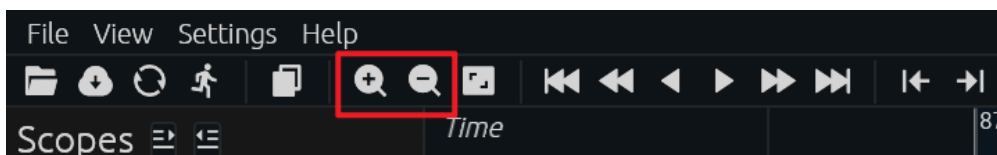
添加信号时，还可以在上图“Variables”字样右侧的“Filter”搜索框内输入信号名称进行搜索。

删除信号时，只需点击上图右侧的某个信号，按下 `Del` 键即可。

3. 波形缩放

Surfer支持两种波形缩放方式，第一种是按住 `^ Ctrl` 键，再滚动鼠标滚轮。此时，Surfer将以鼠标指针指向的竖直线为中心，缩放波形。

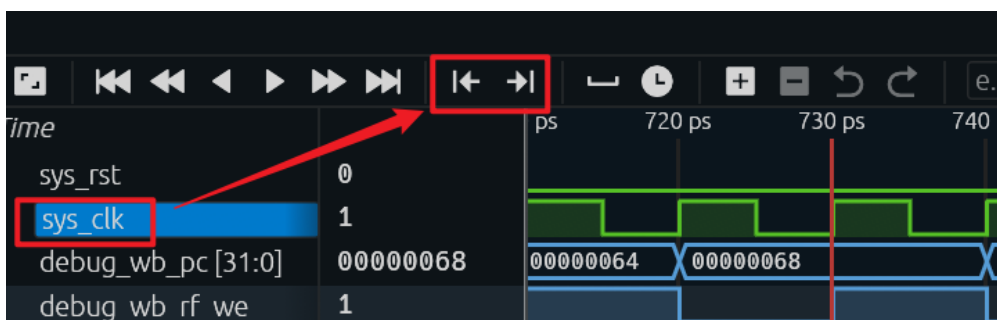
第二种方法是点击工具栏的放大、缩小按钮，如下图所示。



4. 波形定位

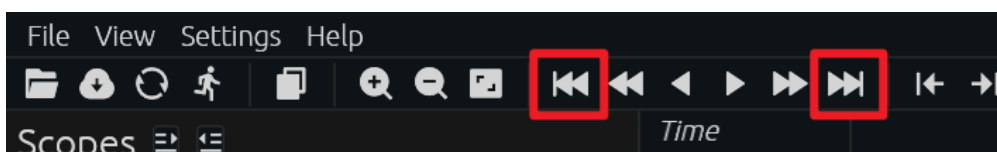
Surfer支持滚动鼠标滚轮来左右移动波形，比Vivado方便许多。但用鼠标点击波形信号线，虽然能够查看信号在对应时刻的值，但不能选中信号。

此外，可以点击选中某个信号之后，点击工具栏的“吸边”工具，从而定位到信号发生变化的上一个或下一个时刻，如下图所示。



点击选中某个信号之后，也可以通过键盘的 `← Left` 和 `→ Right` 方向键来定位信号发生变化的时刻。

点击菜单栏的“Go to start”或“Go to end”按钮，可快速定位到波形的最开始或最末尾，如下图所示。



5. 波形设置

在某个信号上右键，可设置波形的显示格式、颜色、设置分组等等，如下图所示。

